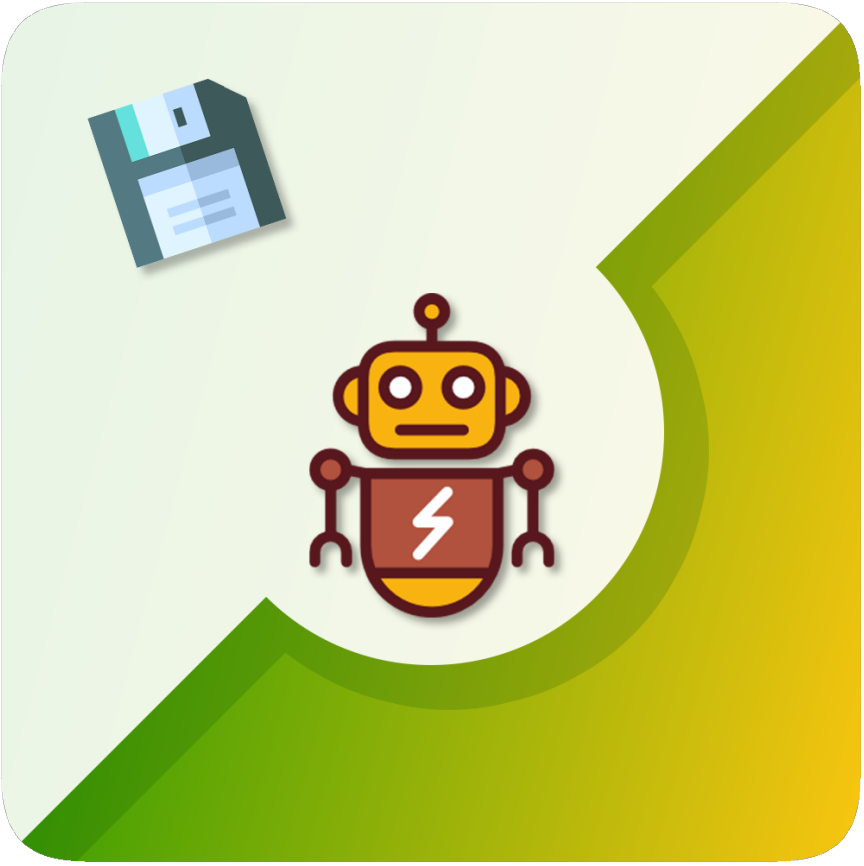

Simple Complex SaveGame Util

By: Paul Willy Hammer
February 28, 2019
Version: 1.0.0.0

TABLE OF CONTENTS

1. ARTICLES **3**

1.1. About the Library 3

1.2. Getting Started 4

1.3. Frequently Asked Questions 8

1.4. Support 8

2. API DOCUMENTATION **9**

2.1. SaveGameUtil Class 9

2.1.1. Setup *9*

2.1.1.1. Init 9

2.1.1.2. RegisterSaveFileModel 10

2.1.1.3. SetSaveFilesSubFolder 10

2.1.1.4. SetSaveFilesExtension 11

2.1.1.5. GetFullSaveFilePath 11

2.1.1.6. GetSaveGameName and Exists 12

2.1.1.7. Delete 12

2.1.2. Load - Save *13*

2.1.2.1. Save 13

2.1.2.2. SaveAsync 14

2.1.2.3. Load 15

2.1.2.4. LoadAsync 16

2.1.3. Encryption *17*

2.1.3.1. EnableEncryption 17

2.1.3.2. DisableEncryption 17

2.1.3.3. IsEncryptionEnabled 18

2.2. Further information on use 19

2.3. The Savegame Data Model 19

1. Articles

1.1.About the Library

The SimpleComplex SaveGame Util allows you to load, save, encrypt and decrypt very complex and large data models. These data models can contain lists and dictionaries with complex data types and circular references. In addition to synchronous save/load, asynchronous load/save is also supported.

The library was developed to allow a very fast integration into your project.

For this reason e.g. the Newtonsoft.Json.dll was embedded into the library. This was done to avoid any version conflicts with other libraries you use that use Newtonsoft Json.

The library provides an accessible class called SaveGameUtil. This class loads, stores and encrypts your saved games.

1.2.Getting Started

Developed to eliminate the problems of storing complex savegames.

In the following steps you will learn how to proceed, what you need to do to achieve a smooth integration into your project.

The asset contains a demo. It consists of an example script that illustrates how to use SaveGameUtil using a complex data model. There is also a demo scene that shows how to save, load and delete the saved data.

If you have already downloaded and installed the asset, you can skip step 1.

1. Download the asset from the Asset Store and import it into your project.
2. Add the *namespace* named **PWHSoftSaveGame** to your class.

```
using PWHSoftSaveGame;
```

   If you have not yet created a **MonoBehaviour** script, create one.

3. Create the Start() method in your script and call the *Init()* method of **SaveGameUtil**.

   The *Init()* method must always be executed from a main thread.

```
// Start is called before the first frame update
void Start()
{
    //Init the save game util.
    SaveGameUtil.Init();
}
```

4. Create a model in which you want to save your savegame data.

```
public class SaveGameModel
{
    public string LocalPlayerName { get; set; }
    public Date FirstAppStartDate { get; set; }

    public List<ScoreModel> Scores
    {
        get;
        set;
    }

    public class ScoreModel
    {
        private DateTime Date { get; set; }
        private long Score { get; set; }
    }
}
```

5. Register your model. The best way to do this is to use the *Init()* method within the Start method.

```
// Start is called before the first frame update
void Start()
{
            //Init the save game util.
            SaveGameUtil.Init();

            //Register the save game model
            SaveGameUtil.RegisterSaveFileModel<SaveGameModel>("svg1");
}
```

Info: With this step you have already set up everything that you can load and save saved data.

6. Now you can check whether the player has already played the game by trying to load the game.
   If the result is zero, the player has never played the game before and you can create and save a new save level.

   It is best to define a new method called "*Load*" in your script. You can then call this method via the *start* method

after the *RegisterSaveFileModel*.

```
// Start is called before the first frame update
void Start()
{
        //Init the save game util.
        SaveGameUtil.Init();

        //Register the save game model
        SaveGameUtil.RegisterSaveFileModel<SaveGameModel>("svg1");

        Load();

}

public void Load()
{
    _saveGame = SaveGameUtil.Load<SaveGameModel>();

    if (_saveGame == null)
    {
        _saveGame = new SaveGameModel
        {
            Scores = new List<SaveGameModel.ScoreModel>(),
            FirstAppStartDate = DateTime.UtcNow,
            LocalPlayerName = "TestPlayer1"
        };

        //Now save the new game data
        SaveGameUtil.Save(_saveGame);
    }
}
```

7. If, for example, you have added a new score to the data model after a game session, you should save the data. We recommend that you save the game asynchronously to ensure a smooth gaming experience for your customers. You should also write this in a public method within your script, as you can also call this function from the UI.

```
public void SaveTheGame()
{
    //Save the save game async
    SaveGameUtil.SaveAsync(
        _saveGame,
        () => Debug.Log("The savegame was saved."),
        exception => Debug.LogError(exception.Message));
}
```

These steps have illustrated the easy handling of SaveGameUtil. However, this tool offers many more possibilities such as encrypting the savegame (see chapter 2.1.3.1). Further information about the functionality can be found in the demo project.

1.3.Frequently Asked Questions

Since this is the first document version and no questions have been asked yet, they will be updated with the next version.

1.4.Support

If you have any questions, please contact us with name and subject "Request SimpleComplexSaveGame Support" to the following e-mail address:

support.unityassetstore@pwhsoft.de

I will answer your e-mail as soon as possible. Please understand that this may take some time.

2. Api Documentation

2.1.SaveGameUtil Class

2.1.1.Setup

2.1.1.1. Init

The *Init* method initializes the **SaveGameUtil**. This method my be executed once before calling other functions of this class. This method should be executed from the main thread. We recommend to call this method at the start of the application.

The parameter *applicationPath:string* is optional. It sets the default path where e.g. memory files are located. If this is not set, the default value of *Application.PersistentPath* is used.

If the *applicationPath* directory does not exist, it will be created.

Examples

```
//Init the savegame util.
SaveGameUtil.Init();
```

```
//Init the savegame util.
SaveGameUtil.Init(applicationPath:"C:\\Test\\");
```

2.1.1.2. RegisterSaveFileModel

This method is used to register a saved game model and must be executed before actions such as loading and saving are called.

Examples

```
//Register the savegame class
SaveGameUtil.RegisterSaveFileModel<TestSaveGameModel>("mainsavegame");
```

```
//Register the savegame class
SaveGameUtil.RegisterSaveFileModel("mainsavegame", typeof(TestSaveGameModel));
```

2.1.1.3. SetSaveFilesSubFolder

Specifies the name of the subfolder in which the storage files are stored.

The default folder name is "*savefiles*". If the subfolder is set to zero or empty, the default subfolder is the persistent application path of the *Init()* method.

Example

```
//Sets the subfolder within the Persistent application path.
SaveGameUtil.SetSaveFilesSubFolder(Path.Combine("savegames","main"));
```

2.1.1.4. SetSaveFilesExtension

Sets the file extension of the memory files.

The default file extension is *".savegame"*. If the argument named *extension* is set to zero or empty, the default file extension is used.

Example

```
//Sets the file extension for the savegames.
SaveGameUtil.SetSaveFilesExtension("secretsavefile");
```

2.1.1.5. GetFullSaveFilePath

Returns the full storage path for the respective data type.

Note: If the type has not been registered, an error message will be thrown.

Examples

```
SaveGameUtil.GetFullSaveFilePath<TestSaveGameModel>();
```

```
SaveGameUtil.GetFullSaveFilePath(typeof(TestSaveGameModel));
```

2.1.1.6. GetSaveGameName and Exists

Returns the savegame name for a model type.

Examples

```
//Get the savegame name
var saveGameName = SaveGameUtil.GetSaveGameName<TestSaveGameModel>();

//Check for savegame exists
if (saveGameName == null)
{
    Debug.Log("No savegame...");
}
else
{
    Debug.Log("The savegame name for the model " + nameof(TestSaveGameModel)
+ " is " + saveGameName);
}
```

```
var saveGameName =
SaveGameUtil.GetSaveGameName(typeof(TestSaveGameModel));
```

2.1.1.7. Delete

If a savegame file exists for the type, it will be deleted.

Example

```
//Delete SaveGame
SaveGameUtil.Delete<TestSaveGameModel>();
```

2.1.2. Load - Save

2.1.2.1. Save

Saves a save model in a file. The process runs synchronously. If a password has been set, the file is stored encrypted.

Parameter "saveGameModel": The savegame model to save to file.

Type Parameter “TModel": The save model type for which the object should be loaded.

NullReferenceException: Occurs if the save file model was not registered.

Example

```
//Saves the savegame model
SaveGameUtil.Save(saveGameModel);
```

2.1.2.2. SaveAsync

Saves a save model in a file. The process runs asynchronously. If a password has been set, the file is stored encrypted.

Parameter "saveGameModel": The savegame model to save to file.

Parameter "onComplete": If not null, this function is called after saving successfully.

Parameter "onFailure": If an onFailure action is set and an exception occurs, *n*o exception is thrown, but the onFailure function returns it as argument.

Type Parameter "TModel": The save model type for which the object should be loaded.

NullReferenceException: Occurs if the save file model was not registered.

Example

```
//Save the savegame async
SaveGameUtil.SaveAsync(
            saveGame,
            () => Debug.Log("The savegame was saved."),
            exception => Debug.LogError(exception.Message));
```

```
//Save the savegame async
SaveGameUtil.SaveAsync(saveGame);
```

2.1.2.3. Load

Loads a savegame.The process is performed synchronously. If a password has been set, an attempt is made to decrypt the file with the password. If the password is wrong, an exception will be thrown.

Type Parameter “TModel": The save model type for which the object should be loaded.

NullReferenceException: Occurs if the save file model was not registered.

Example

```
//Load the savegame
SaveGameUtil.Load<TestSaveGameModel>();
```

2.1.2.4. LoadAsync

Loads a savegame. The process is performed asynchronously. If a password has been set, an attempt is made to decrypt the file with the password. If the password is wrong, an exception will be thrown.

Parameter "onComplete": The onComplete function must not be null. It is called when the save file is loaded. Note, however, that the value can be zero if no save file existed.

The argument returns the loaded savegame model, if exists.

Parameter "onFailure": If an onFailure action is set and an exception occurs, no exception is thrown, but the onFailure function returns it as argument.

Type Parameter "TModel": The save model type for which the object should be loaded.

NullReferenceException: Occurs if the save file model was not registered.

Examples

```
//Load savegame async
SaveGameUtil.LoadAsync<TestSaveGameModel>(
loadedSaveGame =>
{
        _saveGameModel = loadedSaveGame;
        Debug.Log("The savegame was loaded");
},
onFailure: exception =>
{
        Debug.LogError(exception.Message);
});
```

```
//Load savegame async
_saveGameModel = await SaveGameUtil.LoadAsync<TestSaveGameModel>(onFailure: exception =>
{
   Debug.LogError(exception.Message);
});
```

2.1.3. Encryption

2.1.3.1. EnableEncryption

Enables encryption of save files using a password.

The parameter named password is used to set the password to encrypt the savegame files.

The password is not used to encrypt the original but its sha256 value is used for. The function returns a boolean true value if the encryption was enabled.

If the parameter named password is set to null then the encryption will be disabled.

Example

```
//Enable Encryption
var success = SaveGameUtil.EnableEncryption("123456");
```

2.1.3.2. DisableEncryption

Disables encrypting files by password feature.

Example

```
//Disable Encryption
SaveGameUtil.DisableEncryption();
```

2.1.3.3. IsEncryptionEnabled

Describes a get only property that returns true if a password has been set to encrypt or decrypt the savegames.

Example

```
//Check encryption is enabled
if (SaveGameUtil.IsEncryptionEnabled)
    Debug.Log("The savegame is protected.");
```

2.2.Further information on use

2.3.The Savegame Data Model

The savegame data model can be complex. However, it may only consist of properties. Instead of arrays (e.g. string[]) lists (List<string>) should be used.

You can use normal data fields and arrays if they are assigned the attribute [JsonIgnore].

To be able to use the names of your savegame without problems after renaming a variable in your datamodel, you should assign a fixed name with the Json Attribute [JsonProperty("Name")].